



EPICODE

Pratica 2

Felipe Amorim Soria

Obiettivo:

Analizzare un codice Python al fine di migliorare le analisi critiche, verificare possibili errori e suggerire miglioramenti.

Indice:

1. Analisi del codice
2. Identificazione degli scenari non contemplati e
Analisi dei possibili errori
3. Conclusione

1. Analisi del codice

```
import datetime
def assistente_virtuale(comando):
    if comando == "Qual è la data di oggi?":
        oggi = datetime.datetime.today()
        risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
    elif comando == "Che ore sono?":
        ora_attuale = datetime.datetime.now().time()
        risposta = "L'ora attuale è " + ora_attuale.strftime("%H%M")
    elif comando == "Come ti chiami?":
        risposta = "Mi chiamo Assistente Virtuale"
    else:
        risposta = "Non ho capito la tua domanda."
        return risposta
while True
comando_utente = input("Cosa vuoi sapere? ")
    if comando_utente.lower() == "esci":
        print("Arrivederci!")
        break
    else:
        print(assistente_virtuale(comando_utente))
```

1. Analisi del codice

- Il codice presentato contiene uno script su un'IA o un assistente virtuale scritto in linguaggio **Python**.
- Il codice inizia importando il dizionario **datetime**, la cui funzione è fornire classi e funzioni per lavorare con date, orari, fusi orari e operazioni correlate.

1. Analisi del codice

- Allo stesso tempo, include un breve dialogo tra l'utente e la macchina.
- Alla fine dello script, è presente la funzione **while**, che rende la "conversazione" un ciclo continuo.

2. Identificazione degli scenari non contemplati e Analisi dei possibili errori

Il codice può presentare alcuni scenari in cui si verificheranno bug:

1. Il codice non fornisce l'opzione per la funzione incorporata "input", la quale viene utilizzata per catturare l'input dell'utente e creare così un dialogo tra utente/macchina.

2. Identificazione degli scenari non contemplati e Analisi dei possibili errori

2. Il codice richiede che classifichiamo a quale classe appartengano sia la domanda che la risposta.

Se ciò non viene fatto, automaticamente il codice non funzionerà.

Come per esempio:

- **int**: una classe per rappresentare numeri interi.
- **str**: una classe per rappresentare stringhe (testi).
- **float**: una classe per rappresentare numeri decimali (punto flottante).
- **bool**: per valori booleani.

2. Identificazione degli scenari non contemplati e Analisi dei possibili errori

3. C'è un errore di digitazione in **datetime.datetoday()**.
La forma corretta sarebbe: **datetime.datetime.today()**
per ottenere la data attuale.
4. C'è l'assenza di : dopo **while True**, poiché senza i
due punti il codice restituirà un errore.

3. Conclusione

L'analisi del codice fa parte della quotidianità di un hacker etico, ed è essenziale per identificare vulnerabilità e comportamenti inaspettati nei programmi. La capacità di leggere e comprendere cosa ogni input genera come output è fondamentale, poiché più rapida e precisa è questa analisi, più efficiente sarà l'automazione dei processi e l'individuazione di errori.

3. Conclusione

Inoltre, un'analisi attenta consente di comprendere con maggiore facilità come un codice può essere manipolato, il che è cruciale, specialmente quando si tratta di identificare malware in linguaggi come Python o altri. Sviluppare queste competenze di osservazione critica e risoluzione dei problemi rafforza la pratica della sicurezza informatica e la prevenzione degli attacchi.